```
    init(name: String, age: Int) {
        self.name = name
        self.age = age
    }

    func display() {
       print("Name: \(name), Age: \(age)")
    }
}

let person = Person(name: "Alice", age: 30)
person.display()
```

Writing Your First Swift Program

Let's write a simple Swift program that asks the user for their name and age, then displays a greeting message and calculates the year they were born.

1. **Create a New Xcode Project:**
   - Open Xcode and select "File" > "New" > "Project".
   - Choose a template for your project (e.g., "iOS App") and click "Next".
   - Enter the project details, including the name and organization identifier, and click "Next".
   - Choose a location to save your project and click "Create".

2. **Set Up the User Interface:**
   - Open the Main.storyboard file.
   - Drag a Label, Text Field, and Button from the Object Library to the View Controller scene.
   - Arrange the UI elements and set their properties (e.g., placeholder text for the Text Field, title for the Button).

3. **Connect the UI Elements to Code:**
   - Open the Assistant Editor by selecting "View" > "Assistant Editor" > "Show Assistant Editor".
   - Control-drag from the Label, Text Field, and Button to the ViewController.swift file to create outlets and actions.

```
import UIKit
class ViewController: UIViewController {
    @IBOutlet weak var nameTextField: UITextField!
    @IBOutlet weak var ageTextField: UITextField!
    @IBOutlet weak var resultLabel: UILabel!

    @IBAction func greetButtonTapped(_ sender: UIButton) {
        let name = nameTextField.text ?? ""
        let age = Int(ageTextField.text ?? "") ?? 0
        let currentYear = Calendar.current.component(.year, from: Date())
        let birthYear = currentYear - age
        resultLabel.text = "Hello, \(name)! You were born in \(birthYear)."
    }
}
```

4. **Run the Application:**
   - Select a simulator from the toolbar or connect a physical device.
   - Click the "Run" button to build and run your application.
   - Enter your name and age in the text fields, then tap the button to see the result.

Advanced Swift Features

Swift offers a range of advanced features that enable developers to write efficient and expressive code. Some of these features include closures, protocols, extensions, generics, and SwiftUI for building user interfaces.

Closures:

Closures are self-contained blocks of functionality that can be passed around and used in your code. They are similar to blocks in C and lambdas in other programming languages.

```
let names = ["Alice", "Bob", "Charlie"]
let sortedNames = names.sorted { $0 < $1 }
print(sortedNames)
```

Protocols:

Protocols define a blueprint of methods, properties, and other requirements for a particular task or functionality. Classes, structs, and enums can adopt protocols to provide actual implementations.

```
protocol Greetable {
    func greet(name: String) -> String
}

class Greeter: Greetable {
    func greet(name: String) -> String {
        return "Hello, \(name)!"
    }
}

let greeter = Greeter()
print(greeter.greet(name: "Alice"))
```

Extensions:

Extensions add new functionality to an existing class, struct, enum, or protocol type.

```
extension String {
    func reversed() -> String {
        return String(self.reversed())
```